



TÓPICOS EM INFORMÁTICA

Inicialização, módulos e ciclo de vida



CLASSE STARTER (INICIALIZADORA)

- A libgdx é escrita inteiramente dentro de um projeto chamado **core** (gerado pelo instalador da libgdx).
- Todas as plataformas referenciam o código do projeto core. Dito isto, cada projeto por plataforma possui uma **classe starter** (inicializadora) que irá **referenciar o projeto core e inicializá-lo** (disparar para aquele sistema operacional). Em suma:
 - O projeto Android possui uma classe starter.
 - O projeto Desktop possui uma classe starter (DesktopLauncher).
 - O projeto IOS possui uma classe starter.
 - O projeto HTML5 possui uma classe starter.



DESKTOP STARTER

- Desktop starter (ou DesktopLauncher – versão mais atual):

```
public class DesktopLauncher {  
    public static void main(String[] argv) {  
        LwjglApplicationConfiguration config = new LwjglApplicationConfiguration();  
        new LwjglApplication(new MyGame(), config);  
    }  
}
```

- Android starter (pode ter outro nome – igual a anterior):

```
public class AndroidStarter extends AndroidApplication {  
    public void onCreate(Bundle bundle) {  
        super.onCreate(bundle);  
        AndroidApplicationConfiguration config = new AndroidApplicationConfiguration();  
        initialize(new MyGame(), config);  
    }  
}
```



CONFIG

- `config.allowSoftwareMode = false; //default`
 - Caso **true**, ativa a possibilidade de renderizar o programa usando software (CPU) ao invés do OpenGL (caso ele não esteja disponível).
Como isso funciona?
- `config.idleFPS = 60; //default`
 - Seta a **quantidade de frames quando a aplicação não está em primeiro plano**. A CPU vai entrar em sleep quando for necessário. 0 = nunca dorme (CPU), -1 não renderiza nada.
- `config.foregroundFPS = 60; //default`
 - Seta a quantidade de frames quando a aplicação **está em primeiro plano**. 0 = nunca dorme (CPU).



CONFIG

- `config.fullscreen = false; //default`
 - Caso true **inicia a sua aplicação em fullscreen** (tela cheia).
- `config.resizable = true; //default`
 - Caso setado com **false impede a janela da sua aplicação de ser redimensionada.**
- `config.height` e `config.width`;
 - Seta a altura e largura da sua aplicação quando for lançada (em quantidade de pixels).



CONFIG

- `config.title; //default`
 - É uma String e pode receber o título da janela da sua aplicação (quando ela for lançada).
- `config.pauseWhenBackground = false; //default`
 - **Pausa** o jogo quando está em background caso seja setado como true.
- `config.pauseWhenMinimized = false; //default`
 - **Pausa** o jogo quando está minimizado no desktop caso sej setado como true.



CONFIG

- `config.useGL30 = false; //default`
 - Tenta rodar o OpenGL 3.0.
- `config.vSyncEnabled = true; //default`
 - Caso seja setado como false, desativa o Vertical Synchronization (V-Sync).
- O que é V-Sync?



CONFIG

- `config.useGL30 = false; //default`
 - Tenta rodar o OpenGL 3.0.
- `config.vSyncEnabled = true; //default`
 - Caso seja setado como false, desativa o Vertical Synchronization (V-Sync).
- O que é V-Sync?
 - V-Sync é um esquema de sincronização forçado por software para conter os tearings (rasgos) de renderização na tela.

SCREEN TEARING (RASGOS)

COMO RESOLVER TEARING?

TEARING

sim
conserta

V-SYNC OFF

Screen
Tearing

G-SYNC



SINCRONIZAÇÃO

- Qual a diferença entre:
 - V-Sync
 - Free Sync
 - G-Sync



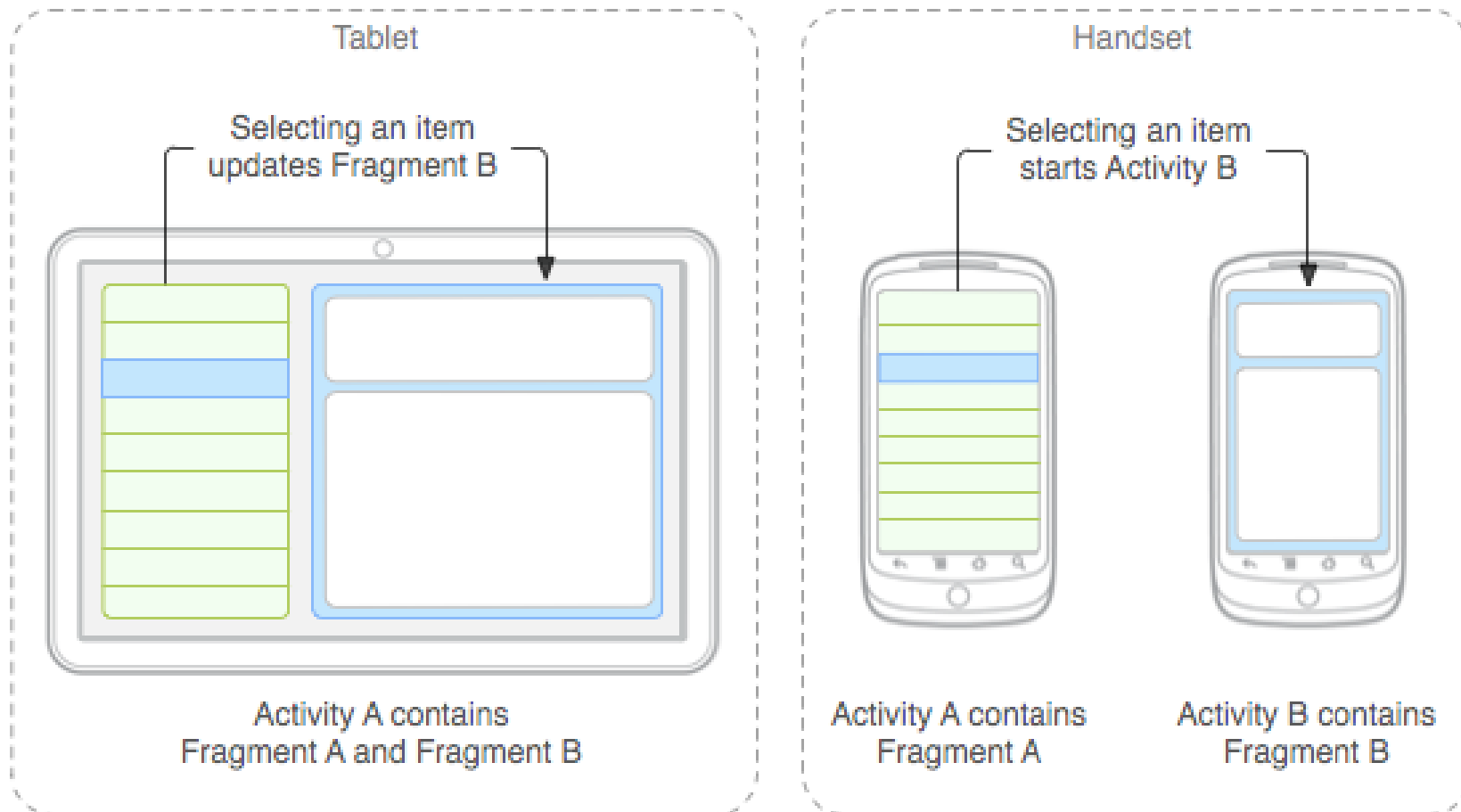
CONFIG

- Método addIcon:
 - Adiciona um ícone para a janela do seu jogo ou aplicação.
 - Exemplo:
 - Você pode jogar uma png dentro da pasta assets da sua aplicação e, posteriormente, executar o seguinte comando no DesktopLauncher / Starter:

```
config.addIcon("icon_128.png", FileType.Internal);
```

FRAGMENTS (ANDROID)

- É possível criar telas com base no Fragments do android usando a libgdx:





ANDROID MANIFEST

- Lembre-se de, caso necessário, editar o AndroidManifest em seu projeto do Android (como por exemplo a versão do Android):

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.me.mygdxgame"
    android:versionCode="1"
    android:versionName="1.0" >

    <uses-sdk android:minSdkVersion="8" android:targetSdkVersion="15" />

    <application
        android:icon="@drawable/ic_launcher"
        android:label="@string/app_name" >
        <activity
            android:name=".MainActivity"
            android:label="@string/app_name"
            android:screenOrientation="landscape"
            android:configChanges="keyboard|keyboardHidden|orientation">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>

</manifest>
```



ANDROID MANIFEST

- Se precisar escrever no SD, acesso à internet, usar o vibrador ou gravar audio, as seguintes permissões precisam ser adicionadas ao Manifest:

```
<uses-permission android:name="android.permission.RECORD_AUDIO"/>  
<uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"/>  
<uses-permission android:name="android.permission.VIBRATE"/>
```

- Lembrando que o usuário pode sempre ficar receoso em relação à quantidade de permissões que você precisa conceder para rodar um aplicativo. Dessa forma, **escolha de maneira inteligente**.

LIVE WALLPAPER

- **Curiosidade:** A libgdx permite que você crie livewallpaper:





DAYDREAMS

- **Curiosidade:** A também permite que você crie DayDreams, que são nada mais do que os antigos “papéis de parede animados”.
- Eles são exibidos como plano de fundo quando o telefone está travado ou em espera.

HTML 5

- Quando você faz o deploy pra HTML5, existe uma tela de loading que carrega todos os assets do disco:



- **Curiosidade:** É possível alterar o layout desse carregamento.



CRONOGRAMA

- Vimos até agora:
 - Inicialização e configuração



ESQUELETO DA APLICAÇÃO

```
public class MyGame implements ApplicationListener {  
    public void create () {  
    }  
  
    public void render () {  
    }  
  
    public void resize (int width, int height) {  
    }  
  
    public void pause () {  
    }  
  
    public void resume () {  
    }  
  
    public void dispose () {  
    }  
}
```

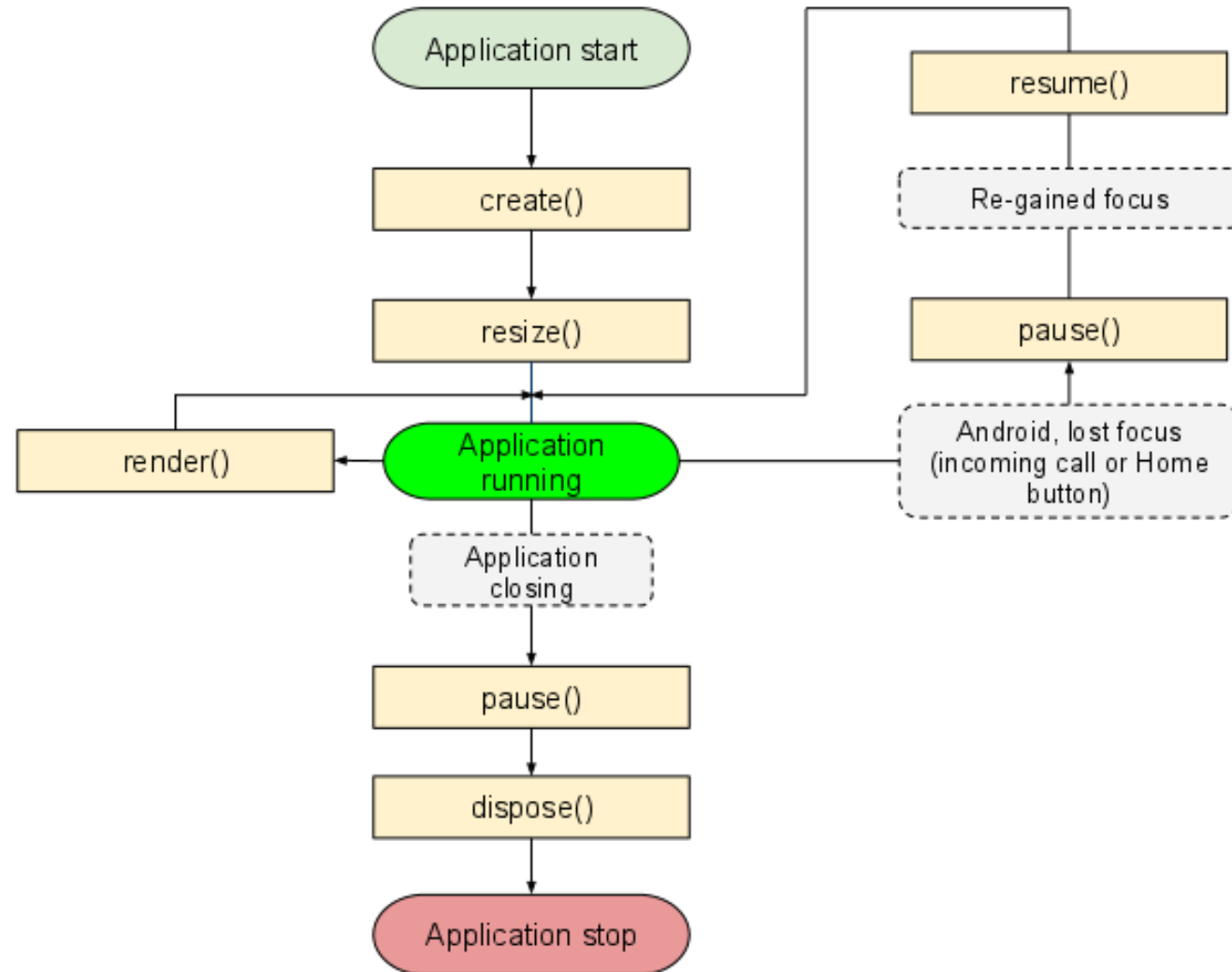
MÉTODOS

- `create()` : O método é chamado uma vez quando a aplicação for criada (quando seu jogo iniciar, o `create` é chamado e tudo que está dentro dele é executado).
- `resize(int width, int height)` : Esse método é chamado toda vez que a tela do jogo for redimensionada e o jogo não está em estado de pausa. Também é chamado uma vez depois do método `create()`.
- `render()` : Game loop. A lógica do jogo e as atividades gráficas todas são normalmente operadas nesse loop. É um loop eterno que dura do início da aplicação até o seu final.

MÉTODOS

- `pause()` : No Android, esse método é chamado quando o botão **home é apertado** (ou uma chamada é recebida). No desktop `pause` só é chamado antes do método `dispose()`.
- `resume()`: Esse método é apenas chamado no Android quando a aplicação volta de um estado pausado.
 - **Nesse método, no android, usualmente é preciso recarregar as texturas e outros recursos.**
- `dispose()` : É chamado quando a aplicação (ou tela) é fechada, finalizada ou “destruída”.

CICLO DE VIDA





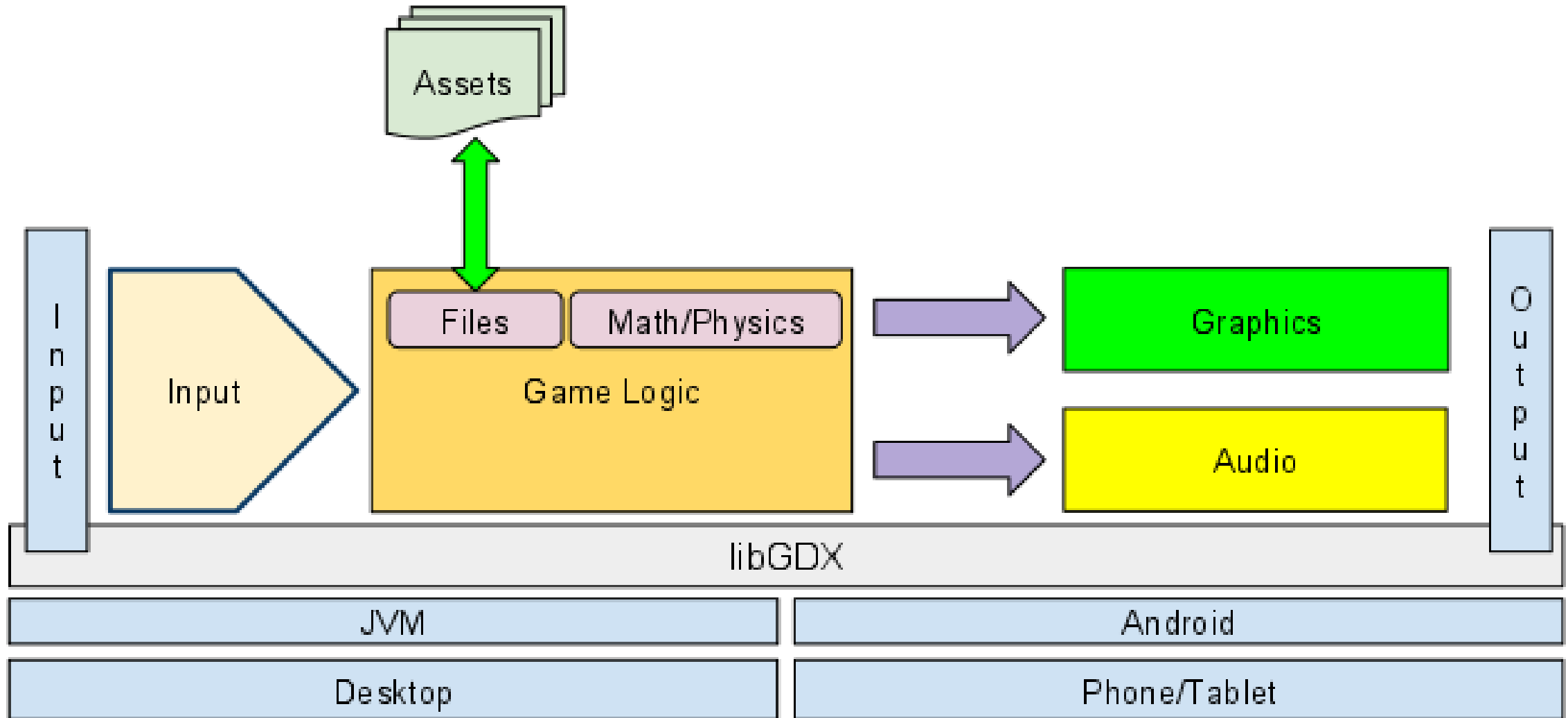
CRONOGRAMA

- Vimos até agora:
 - Inicialização e configuração
 - Ciclo de vida

MÓDULOS

- A Libgdx é composta por:
 - **Application (aplicação)**: Roda a aplicação e informa a API de eventos como redimensionamento de janela. Provê estatísticas de uso de memória, etc.
 - **Files (arquivos)**: Comunicação com arquivos do sistema operacional.
 - **Input (entrada)**: Informa a API sobre eventos originados do mouse, teclado, touch, acelerômetro, etc.
 - **Net (rede)**: Provê meios de acessar recursos por HTTP/HTTPS de forma multi-plataforma, assim como criar servers e clientes TCP.
 - **Audio**: Toca música, efeitos, streaming, etc. Serve também para microfone.
 - **Graphics (gráficos)**: se comunica com o OpenGL, fazendo as requisições de video.

VISÃO GERAL





ACESSANDO OS MÓDULOS

- Exemplo de gravação de som (acesso ao módulo Gdx.audio):

```
final int samples = 44100;
boolean isMono = true;
final short[] data = new short[samples * 5];

final AudioRecorder recorder = Gdx.audio.newAudioRecorder(samples, isMono);
final AudioDevice player = Gdx.audio.newAudioDevice(samples, isMono);

new Thread(new Runnable() {
    @Override
    public void run() {
        System.out.println("Record: Start");
        recorder.read(data, 0, data.length);
        recorder.dispose();
        System.out.println("Record: End");
        System.out.println("Play : Start");
        player.writeSamples(data, 0, data.length);
        System.out.println("Play : End");
        player.dispose();
    }
}).start();
```



MÓDULO INPUT

- O módulo input permite a leitura de diversos tipos de estados de entrada de forma multi-plataforma.
- Permite a leitura de dados do teclado, touchscreen e acelerômetro.
 - No Desktop touchscreen é substituído pelo mouse e acelerômetro não é acessível.

MÓDULO INPUT

- Exemplo de leitura de toque ou clique na tela:

```
if (Gdx.input.isTouched()) {  
    System.out.println("Aconteceu um clique em x=" + Gdx.input.getX() +  
        ", y=" + Gdx.input.getY());  
}
```

- Esse é apenas **uma das formas de tratar inputs** de mouse e teclado, a forma mais correta é utilizando InputProcessors, veremos mais pra frente.

MÓDULO GRAPHICS

- É possível se comunicar **diretamente com o OpenGL** (Libgdx -> LWJGL -> OpenGL). Existem funções que podem chamar **comandos OpenGL dentro da Libgdx**.
- Exemplo para obter a OpenGL API:

```
//retorna a API OpenGL 2.0  
GL20 gl = Gdx.graphics.getGL20 ();  
//Roda o comando para limpar a tela (comando OpenGL).  
gl.glClearColor(1f, 0.0f, 0.0f, 1);  
gl.glClear(GL20.GL_COLOR_BUFFER_BIT);
```

MÓDULO FILES

- O módulo files serve para acessar arquivos internos ou externos ao projeto.
- O exemplo abaixo demonstra acesso a um arquivo interno:

```
Texture myTexture =  
new Texture(  
Gdx.files.internal("assets/textures/brick.png"));
```

MÓDULO AUDIO

- O módulo de audio permite criar e tocar arquivos de música. Ele se comunica diretamente com o hardware de som:

```
Music music = Gdx.audio.newMusic(  
Gdx.files.getFileHandle  
("data/myMusicFile.mp3", FileType.Internal));
```

```
music.setVolume(0.5f);  
music.play();  
music.setLooping(true);
```



MÓDULO NETWORK

- Fornece funções para comunicação de rede (jogos multiplayer), ou jogos que enviem estatísticas pela internet.

```
HttpRequestBuilder requestBuilder = new HttpRequestBuilder();  
HttpRequest httpRequest = requestBuilder.newRequest()  
    .method(HttpMethods.GET)  
    .url("http://www.google.com")  
    .build();  
Gdx.net.sendHttpRequest(httpRequest, httpResponseListener);
```


MÓDULO NETWORK

- Para enviar informação por meio do método HTTP é possível fazer:

```
HttpRequestBuilder requestBuilder = new HttpRequestBuilder();
HttpRequest httpRequest = requestBuilder.newRequest()
    .method(HttpMethods.GET)
    .url("http://www.google.de")
    .content("q=libgdx&example=example")
    .build();
Gdx.net.sendHttpRequest(httpRequest, httpResponseListener);
```



CRONOGRAMA

- Vimos até agora:
 - Inicialização e configuração
 - Ciclo de vida
 - Módulos da aplicação



TIPO DE DISPOSITIVO

- Caso seja necessário (bem raro na minha experiência pessoal – nunca usei na verdade), é possível utilizar códigos específicos para cada tipo de dispositivo. Exemplo:

```
switch (Gdx.app.getType()) {  
    case Android:  
        // android specific code  
        break;  
    case Desktop:  
        // desktop specific code  
        break;  
    case WebGL:  
        // HTML5 specific code  
        break;  
    default:  
        // Other platforms specific code  
}
```

VERSÃO

- De forma similar, também é possível pegar a versão do Android:

```
int androidVersion = Gdx.app.getVersion();
```

- Por fins de debug, também é possível acessar informações do sistema e da JVM, como:

```
long javaHeap = Gdx.app.getJavaHeap();  
long nativeHeap = Gdx.app.getNativeHeap();
```



LOG

- É possível utilizar o log da Libgdx de 3 formas:

```
Gdx.app.log("MyTag", "my informative message");
```

```
Gdx.app.error("MyTag", "my error message", exception);
```

```
Gdx.app.debug("MyTag", "my debug message");
```

- Dependendo da plataforma, as mensagens são salvas em **diferentes ambientes**:
 - Desktop → console
 - Android → LogCat
 - GWT (HTML5) → text area da GwtApplicationConfiguration



LOG

- É possível filtrar para salvar somente as mensagens de log que sejam interessantes para você naquele momento:

```
Gdx.app.setLogLevel(logLevel);
```

- Onde:
 - Application.LOG_NONE: muta todos os logs.
 - Application.LOG_DEBUG: loga todas as mensagens.
 - Application.LOG_ERROR: loga apenas as mensagens de erro.
 - Application.LOG_INFO: loga erros e mensagens normais.

EXERCÍCIO

- Brinque com as alterações de disparo (config) **vistas nessa aula.**
- **Coloque o FPS** de seu jogo para aparecer no título da janela da aplicação.
- Faça um jogo que dispare (**movimente** para qualquer lugar da tela) alguma textura **após o clique na tela ou em um botão da tela.**
- Depois de terminar o primeiro ponto, **monte uma aplicação em que uma textura se desloca da posição em que estava até a posição que foi clicada.**



EXERCÍCIO

- Altere o exercício anterior para incluir o **conceito de velocidade**. Dois cliques na tela fazem a textura se mover de forma mais rápida e um clique apenas moverá a textura **de forma mais lenta**.